

Structures and Pointers in C

In C, pointers can store the address of a structure variable. This is a fundamental concept for high-performance programming, especially in systems software.

1. Defining a Pointer to Structure

```
struct Student {  
    int roll;  
    float marks;  
};  
  
struct Student s;  
struct Student *ptr = &s; // ptr points to s
```

Note: The pointer `ptr` must be of the same type as the structure `Student`.

2. Accessing Members Using Pointers

When working with pointers to structures, we shift from the **dot (.)** operator to the **arrow (->)** operator.

```
ptr->roll = 1;  
ptr->marks = 90.5;
```

Behind the scenes: `ptr->roll` is an alias for `(*ptr).roll`. The parentheses are mandatory because the dot operator has higher priority than the dereference operator.

CodeHive

@ Explore

3. Why use Structure Pointers?

Passing a structure by value means copying every single byte of that structure. If a structure has 50 fields, that's heavy!

- **Speed:** Passing a pointer only copies the memory address (usually 4 or 8 bytes).
- **Memory Optimization:** No redundant copies are created on the stack.
- **Direct Modification:** Pointers allow functions to modify the original structure.



4. Passing to Functions

Using pointers as function arguments is the standard way to handle structures in professional C code.

```
void updateMarks(struct Student *s) {
    s->marks = 95.0; // Updates original student
}

int main() {
    struct Student s1 = {1, 85.0};
    updateMarks(&s1); // Pass address
}
```

CodeHive

@ Explore

5. Arrays and Pointer Arithmetic

Just like standard arrays, structure arrays can be traversed using pointer arithmetic.

```
struct Student arr[3];  
struct Student *p = arr;  
  
p->roll = 101;           // Access first element  
(p + 1)->roll = 102;    // Access second element
```

`p + 1` shifts the pointer by `sizeof(struct Student)` bytes.

CodeHive

@ Explore

6. Using Structures with malloc()

Pointers facilitate the creation of structures on the **Heap** during runtime.

```
struct Student *p = malloc(sizeof(struct Student));
if(p != NULL) {
    p->roll = 1;
    free(p);
}
```

This is the foundation for creating Linked Lists and Trees.

CodeHive

@ Explore

Advantages of Structure Pointers ★

- **Efficiency:** Faster function calls due to less data copying.
- **Flexibility:** Essential for dynamic data structures.
- **Hardware Access:** Common in embedded systems to map registers.
- **Scalability:** Ideal for handling large databases of records.

CodeHive

Ⓢ Explore

Problem

Key Points (Exam Important)

- Use `->` with pointers; `.` with variables.
- **`(*ptr).member`** is logically identical to **`ptr->member`**.
- Pointers are used to pass structures by reference.
- This technique is mandatory for data structures like Linked Lists.

Excellence in Code with CodeHive

CodeHive

@ Explore